

Module 2

Relational Model and E-R Modeling

Topic

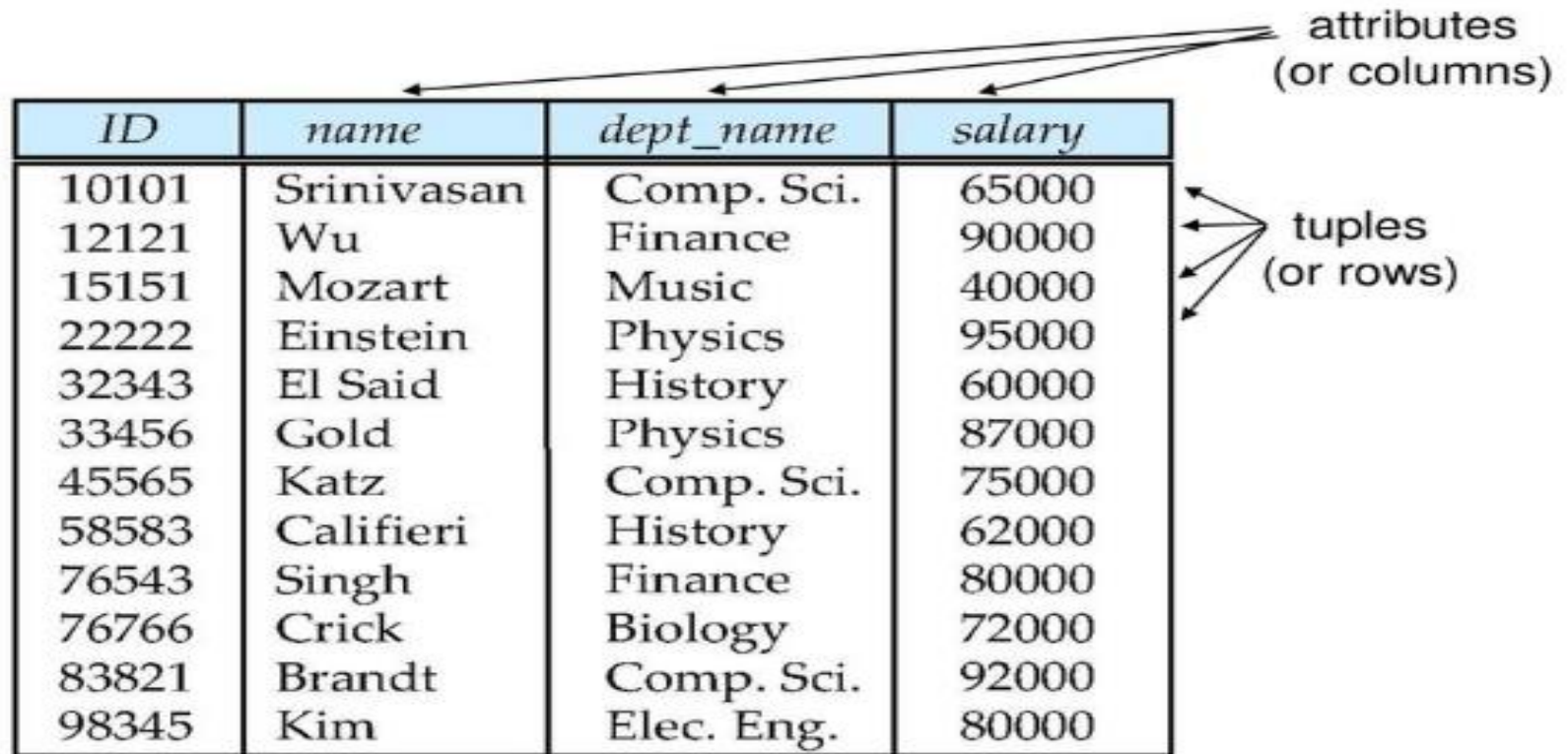
Relational Model

(Abraham Silberschatz)

Relational Model

Example of a Relation

Instructor Relation



The diagram shows a table representing the 'Instructor Relation'. The table has four columns: *ID*, *name*, *dept_name*, and *salary*. The first row is the header, and the subsequent 12 rows are data. Annotations include arrows pointing from the text 'attributes (or columns)' to each of the four column headers, and arrows pointing from the text 'tuples (or rows)' to each of the 12 data rows.

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

A relational database consists of a collection of **tables**, each of which is **assigned a unique name**.

Relational Model

- In general, a *row in a table represents a relationship among a set of values.*
- Since a table is a collection of such relationships, there is a close correspondence between the concept of *table and the mathematical concept of relation*, from which the relational data model takes its name.
- In mathematical terminology, a *tuple is simply a sequence (or list) of values.*
- A relationship between n values is represented mathematically by an *n -tuple of values, i.e., a tuple with n values, which corresponds to a row in a table.*

Relational Model

- Thus, in the relational model the term **relation is used to refer to a table**, while the term **tuple is used to refer to a row**. Similarly, the term **attribute refers to a column** of a table.
- We use the term **relation instance to refer to a specific instance of a relation**, i.e., containing a specific set of rows. The instance of *instructor* has 12 tuples, corresponding to 12 instructors.

Relations are Unordered

- Order of tuples is irrelevant (tuples may be stored in an arbitrary order)
- Example: *instructor* relation with unordered tuples

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Relation

- The order in which tuples appear in a relation is irrelevant, since a relation is a *set of tuples*.
- ***Thus, whether the tuples of a relation are listed in sorted order, or are unsorted, does not matter;*** the relations in the two figures are the same, since both contain the same set of tuples.
- For ease of exposition, we will mostly **show the relations sorted by their first attribute.**

Relation

- For each attribute of a relation, there is a **set of permitted values, called the domain of that attribute.**
- Thus, the **domain of the *salary attribute of the instructor relation* is the set of all possible salary values**, while the domain of the *name* attribute is the set of all possible instructor names.
- We require that, for all relations *r*, *the domains of all attributes of r be atomic.*
- **A domain is atomic if elements of the domain are considered to be indivisible units.**
- For example, suppose the table *instructor* had an attribute *phone number*, which can store a set of phone numbers corresponding to the instructor. Then the domain of *phone number* would not be atomic

Relation

- The **null value is a special value that signifies that the value is unknown or does not exist.**
- For example, suppose as before that we include the attribute *phone number in the instructor relation.*
- *It may be that an instructor does not have a phone number at all, or that the telephone number is unlisted.*
- We would then have to use the **null value to signify that the value is unknown or does not exist.**
- We shall see later that null values cause a **number of difficulties when we access or update the database,** and thus should be eliminated if at all possible

Database Schema

- We must differentiate between the **database schema, which is the logical design of the database, and the database instance, which is a snapshot of the data in the database at a given instant in time.**
- The concept of a relation corresponds to the programming-language notion of a variable, **while the concept of a relation schema corresponds to the programming-language notion of type definition.**
- Although it is important to know the difference between a relation schema and a relation instance, **we often use the same name, such as *instructor*, to refer to both the schema and the instance.**
- Where required, we explicitly refer to the schema or to the instance, **for example “the *instructor schema*,” or “an instance of the *instructor relation*.”**

Database Schema

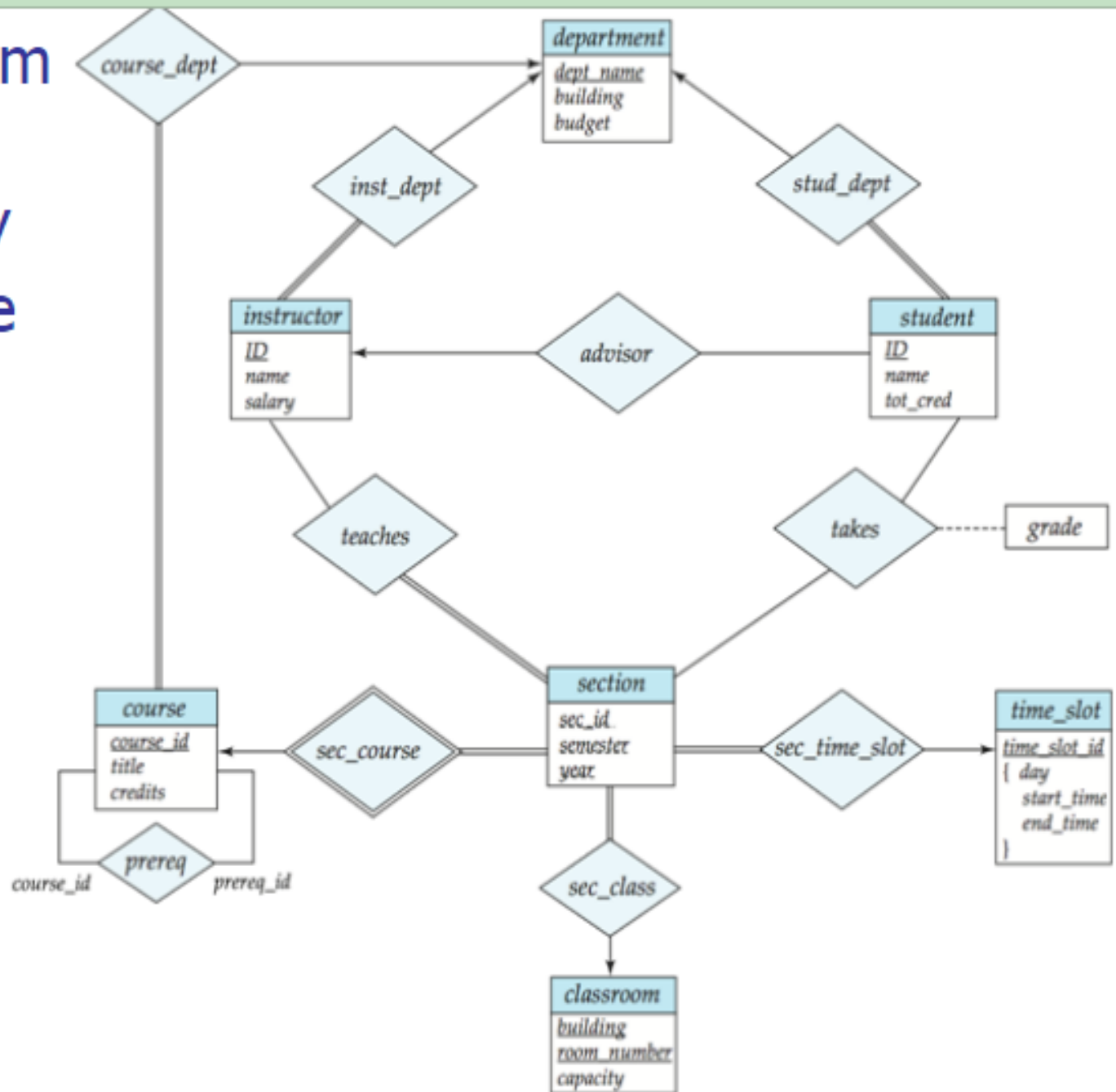
<i>dept_name</i>	<i>building</i>	<i>budget</i>
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

Figure 2.5 The *department* relation.

The schema for that relation is

department (dept name, building, budget)

E-R Diagram for a University Enterprise



- We need a relation to describe the association between instructors and the class sections that they teach. The relation schema to describe this association is
- *teaches (ID, course id, sec id, semester, year)*

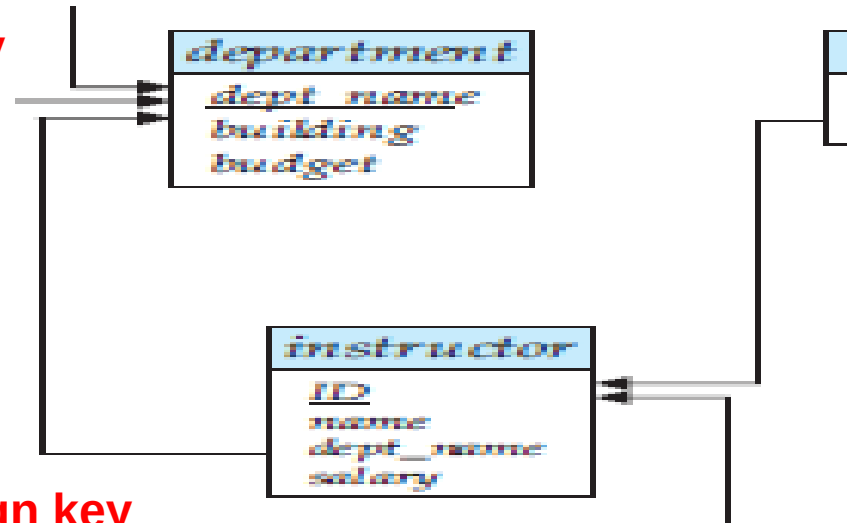
<i>ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

Figure 2.7 The *teaches* relation.

Foreign key

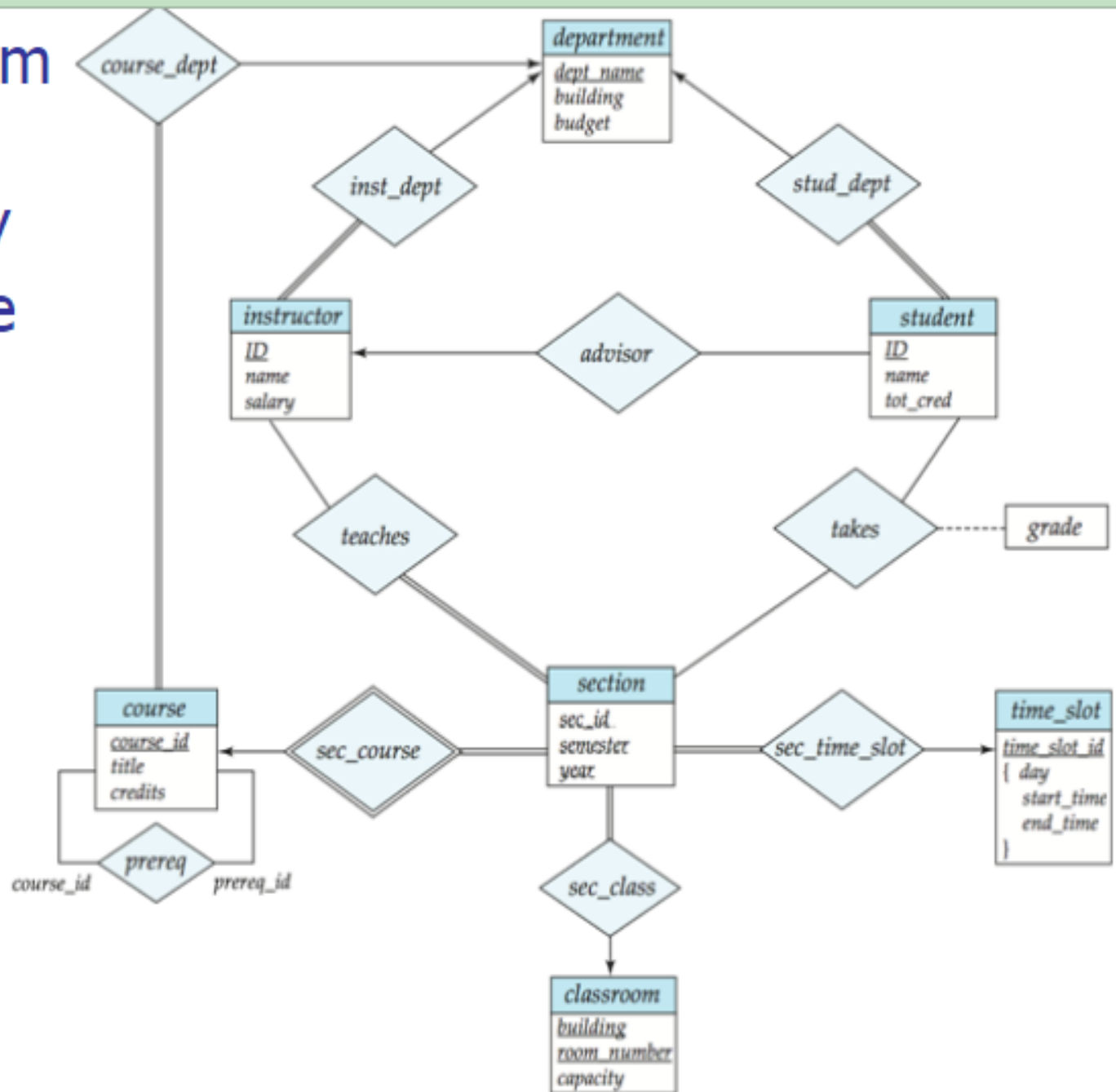
- A relation, say $r1$, may include among its attributes the primary key of another relation, say $r2$.
- This attribute is called a **foreign key from $r1$, referencing $r2$** . The relation **$r1$ is also called the referencing relation of the foreign key dependency**, and **$r2$ is called the referenced relation of the foreign key**.
- For example, the attribute **dept name in instructor** is a **foreign key from instructor**, referencing department, since dept name is the primary key of department.

r2 relation, dept_name is primary



r1 relation, dept_name is foreign key

E-R Diagram for a University Enterprise



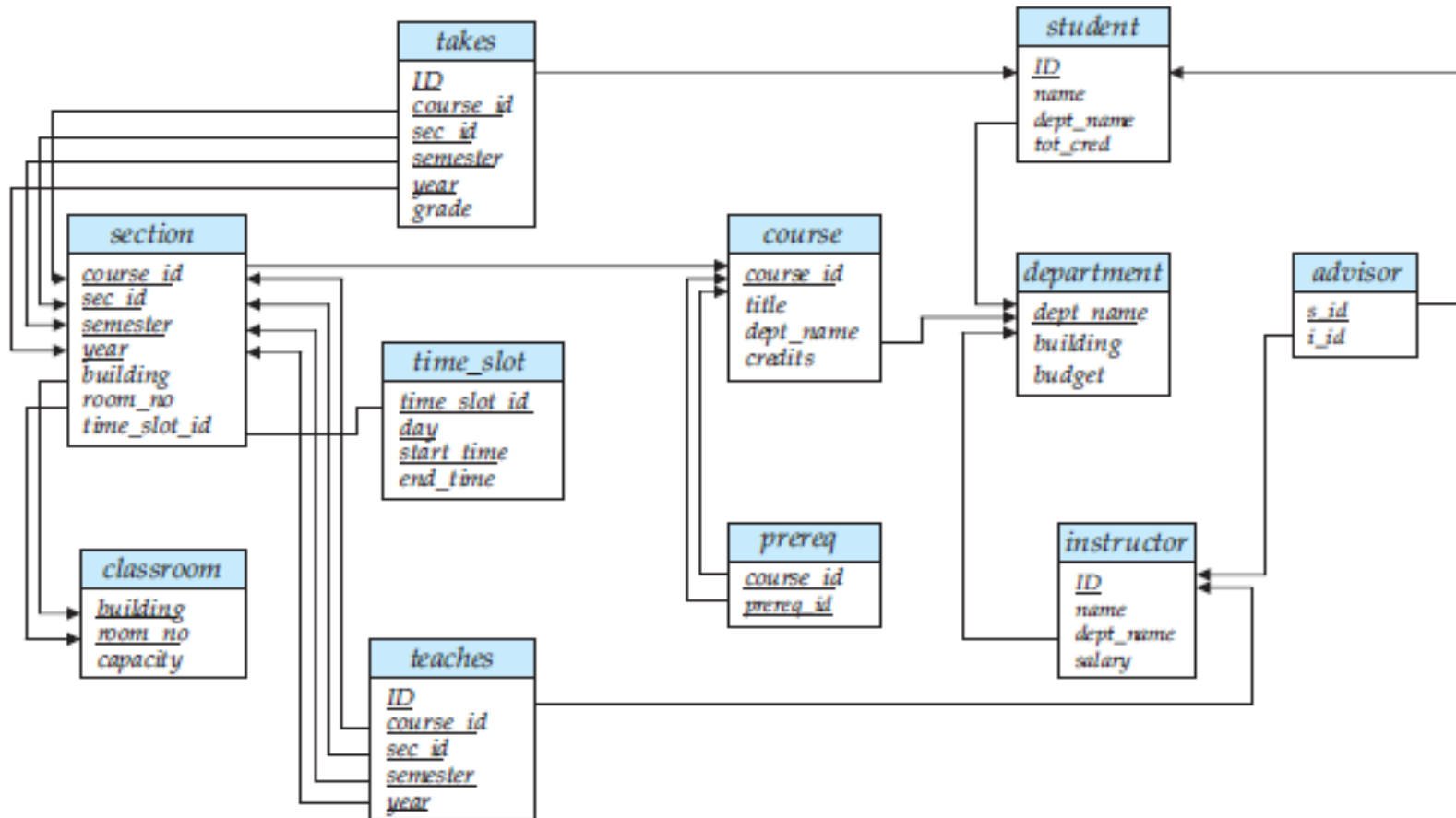


Figure 2.8 Schema diagram for the university database.

Relational Query Languages

- A query language is a language in which **a user requests information from the database.**
- These languages are usually on a level higher than that of a standard programming language. Query languages can be categorized as either **procedural or nonprocedural.**
- In a **procedural language**, the user instructs the system to perform a sequence of operations on the database to compute the desired result.
- In a **nonprocedural language**, the user describes the desired information without giving a specific procedure for obtaining that information.
- There are a number of **“pure” query languages**: The **relational algebra** is procedural, whereas the **tuple relational calculus** and **domain relational calculus** are nonprocedural.

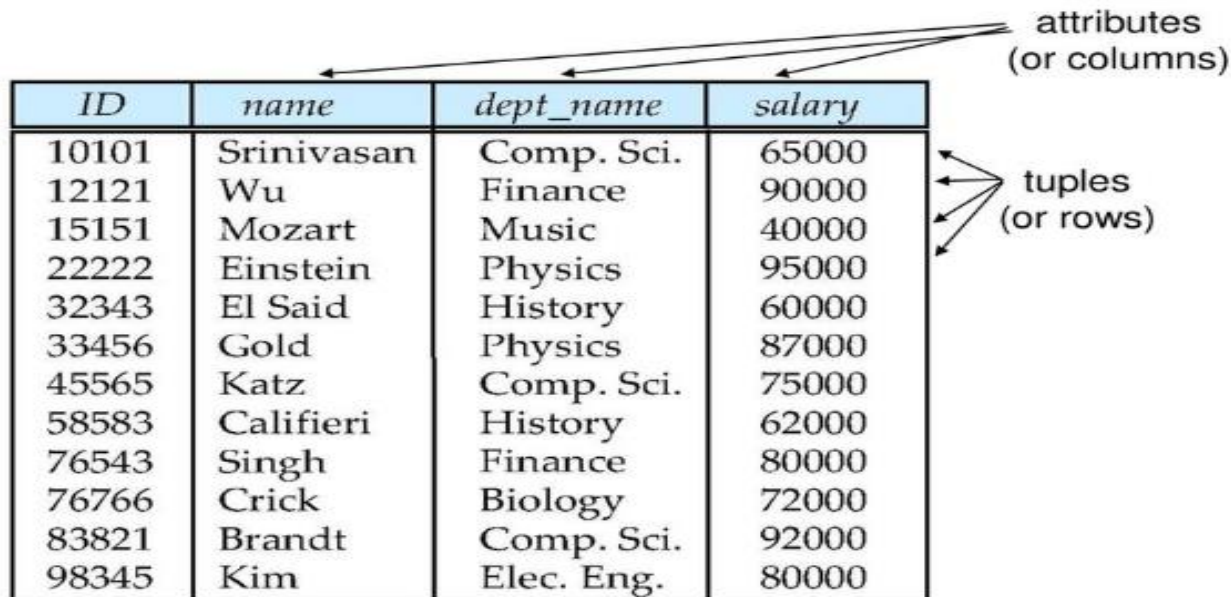
Relational Algebra

- Procedural language
- Six basic operators
 - select
 - project
 - union
 - set difference
 - Cartesian product
 - rename
- The operators take one or more relations as inputs and give a new relation as a result.

Select

- The most frequent operation is the **selection of specific tuples from a single relation** (say *instructor*) that **satisfies some particular predicate** (say $\text{salary} > \$85,000$).

Example of a Relation



The diagram shows a table representing a relation. Above the table, the text 'attributes (or columns)' has three arrows pointing to the column headers: 'ID', 'name', and 'dept_name'. To the right of the table, the text 'tuples (or rows)' has four arrows pointing to the first four rows of data.

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
12121	Wu	Finance	90000
22222	Einstein	Physics	95000
33456	Gold	Physics	87000
83821	Brandt	Comp. Sci.	92000

Figure 2.10 Result of query selecting *instructor* tuples with salary greater than \$85000.

Symbol (Name)

Example of Use

σ

$\sigma_{\text{salary} \geq 85000}(\text{instructor})$

(Selection)

Return rows of the input relation that satisfy the predicate

Select Operation – selection of rows (tuples)

- Relation r

A	B	C	D
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10

- $\sigma_{A=B \wedge D > 5}(r)$

A	B	C	D
α	α	1	7
β	β	23	10

Project



Project Operation – selection of columns (Attributes)

- Relation r :

A	B	C
α	10	1
α	20	1
β	30	1
β	40	2

- $\Pi_{A,C}(r)$

A	C
α	1
α	1
β	1
β	2

 $=$

A	C
α	1
β	1
β	2



Rename Operation

- Allows us to name, and therefore to refer to, the results of relational-algebra expressions.
- Allows us to refer to a relation by more than one name.

Example:

$$\rho_X(E)$$

returns the expression E under the name X

If a relational-algebra expression E has arity n , then

$$\rho_{X(A1, A2, \dots, An)}(E)$$

returns the result of expression E under the name X , and with the attributes renamed to $A1, A2, \dots, An$.

Joined Relations

Cartesian-Product Operation-Example

Relations r, s :

A	B
α	1
β	2

r

C	D	E
α	10	a
β	10	a
β	20	b
γ	10	b

s

$r \times s$:

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

Composition of Operations

- Can build expressions using multiple operations

- Example: $\sigma_{A=C}(r \bowtie s)$

- $r \bowtie s$

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
α	1	α	10	<i>a</i>
α	1	β	10	<i>a</i>
α	1	β	20	<i>b</i>
α	1	γ	10	<i>b</i>
β	2	α	10	<i>a</i>
β	2	β	10	<i>a</i>
β	2	β	20	<i>b</i>
β	2	γ	10	<i>b</i>

- $\sigma_{A=C}(r \bowtie s)$

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
α	1	α	10	<i>a</i>
β	2	β	10	<i>a</i>
β	2	β	20	<i>b</i>

Joined Relations

- Join operations take two relations and return as a result another relation.
- These additional operations are typically used as subquery expressions in the **from** clause
- Join condition – defines which tuples in the two relations match, and what attributes are present in the result of the join.
- Join type – defines how tuples in each relation that do not match any tuple in the other relation (based on the join condition) are treated.

Join Types
inner join
left outer join
right outer join
full outer join

Join Conditions
natural
on <predicate>
using ($A_1, A_2, \dots, A_n$)

Joined Relations – Datasets for Examples

- Relation *loan*

<i>loan-number</i>	<i>branch-name</i>	<i>amount</i>
L-170	Downtown	3000
L-230	Redwood	4000
L-260	Perryridge	1700

- Relation *borrower*

<i>customer-name</i>	<i>loan-number</i>
Jones	L-170
Smith	L-230
Hayes	L-155

- Note: borrower information missing for L-260 and loan information missing for L-155

Joined Relations – Examples

- **loan inner join borrower on**
loan.loan-number = borrower.loan-number

<i>loan-number</i>	<i>branch-name</i>	<i>amount</i>	<i>customer-name</i>	<i>loan-number</i>
L-170	Downtown	3000	Jones	L-170
L-230	Redwood	4000	Smith	L-230

- **loan left outer join borrower on**
loan.loan-number = borrower.loan-number

<i>loan-number</i>	<i>branch-name</i>	<i>amount</i>	<i>customer-name</i>	<i>loan-number</i>
L-170	Downtown	3000	Jones	L-170
L-230	Redwood	4000	Smith	L-230
L-260	Perryridge	1700	<i>null</i>	<i>null</i>

Joined Relations – Examples

- *loan natural inner join borrower*

<i>loan-number</i>	<i>branch-name</i>	<i>amount</i>	<i>customer-name</i>
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith

- *loan natural left outer join borrower*

<i>loan-number</i>	<i>branch-name</i>	<i>amount</i>	<i>customer-name</i>
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-260	Perryridge	1700	<i>null</i>

Joined Relations – Examples

- *loan* **right outer join** *borrower* on
 $loan.loan-number = borrower.loan-number$

<i>loan-number</i>	<i>branch-name</i>	<i>amount</i>	<i>customer-name</i>	<i>loan-number</i>
L-170	Downtown	3000	Jones	L-170
L-230	Redwood	4000	Smith	L-230
null	null	null	Hayes	<i>L-155</i>

- *loan* **natural right outer join** *borrower*

<i>loan-number</i>	<i>branch-name</i>	<i>amount</i>	<i>customer-name</i>
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-155	null	null	Hayes

Joined Relations – Examples

- *loan* **full outer join** *borrower* **using** (*loan-number*)

<i>loan-number</i>	<i>branch-name</i>	<i>amount</i>	<i>customer-name</i>
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-260	Perryridge	1700	<i>null</i>
L-155	<i>null</i>	<i>null</i>	Hayes

Set Operations

- The SQL Set operation is used to combine the two or more SQL SELECT statements.
- Types of Set Operation
 - Union
 - UnionAll
 - Intersect
 - Minus (set difference / except)

Union

- The SQL Union operation is used to **combine the result of two or more SQL SELECT queries.**
- In the union operation, **all the number of datatype and columns must be same in both the tables** on which UNION operation is being applied.
- The union operation **eliminates the duplicate rows from its resultset.**

- **Syntax**

```
SELECT column_name FROM table1
```

```
UNION
```

```
SELECT column_name FROM table2;
```


Example:

The First table

ID	NAME
1	Jack
2	Harry
3	Jackson

The Second table

ID	NAME
3	Jackson
4	Stephan
5	David

Union

- Union SQL query will be:
SELECT * FROM First
UNION
SELECT * FROM Second;
- The resultset table will look like:

ID	NAME
1	Jack
2	Harry
3	Jackson
4	Stephan
5	David

Union All

- Union All operation is equal to the Union operation. It returns the set without removing duplication and sorting the data.

Syntax:

```
SELECT column_name FROM table1  
UNION ALL  
SELECT column_name FROM table2;
```

Example: Using the above First and Second table.

Union All query will be like:

```
SELECT * FROM First  
UNION ALL  
SELECT * FROM Second;
```

The resultset table will look like:

ID	NAME
1	Jack
2	Harry
3	Jackson
3	Jackson
4	Stephan
5	David

Intersect

- It is used to combine two SELECT statements. The Intersect operation returns the common rows from both the SELECT statements.
- In the Intersect operation, the number of datatype and columns must be the same.
- It has no duplicates and it arranges the data in ascending order by default.

Intersect

Syntax

```
SELECT column_name FROM table1  
INTERSECT  
SELECT column_name FROM table2;
```

Example:

Using the above First and Second table.

Intersect query will be:

```
SELECT * FROM First  
INTERSECT  
SELECT * FROM Second;
```

The resultset table will look like:

ID	NAME
3	Jackson

Minus (Except)

- It combines the result of two SELECT statements. **Minus operator is used to display the rows which are present in the first query but absent in the second query.**
- It has no duplicates and data arranged in ascending order by default.

Syntax:

```
SELECT column_name FROM table1  
MINUS  
SELECT column_name FROM table2;
```


Minus (Except)

- **Example**

Minus query will be:

```
SELECT * FROM First  
MINUS  
SELECT * FROM Second;
```

The resultset table will look like:

ID	NAME
1	Jack
2	Harry